

De Gouden Tarwe SQL Database Challenge

(Day 2)

MySQL

Problem Statement

De Gouden Tarwe (Dutch for The Golden Wheat) is a premium bakery chain based in the Netherlands. Established in 2018, the company has grown rapidly and now operates more than 30 branches across 20 major Dutch cities including Amsterdam, Rotterdam, Utrecht, The Hague, Eindhoven, and Groningen.

The bakery sells authentic Dutch baked goods from everyday bread like tijgerbrood and volkorenbrood, to traditional pastries like stroopwafels, appeltaart, bossche bollen, and seasonal specialties like oliebollen (only sold in December) and pepernoten (only sold during the Sinterklaas season in November).

The company employs over 450 people across all branches: bakers, assistant bakers, sales staff, branch managers, delivery drivers, and support staff. It serves tens of thousands of customers, many of whom participate in a loyalty program with tiered benefits (Bronze, Silver, Gold, Platinum).

The Current Problem

De Gouden Tarwe currently stores all its operational data in flat CSV files. As the business has grown, this approach has become unmanageable:

1. Data redundancy: The same product name, price, and description are repeated in every transaction record.
2. No formal relationships: There is no way to enforce that a branch referenced in an order actually exists.
3. No data integrity: A typo in a city name creates a new city that does not actually exist.
4. Impossible to answer business questions: Simple queries like What is our best-selling product in Rotterdam? require manually cross-referencing multiple files.
5. Performance problems: With millions of rows, opening and searching CSV files is painfully slow.

Progress – Day 2

On day 2, we created the database and tables using MySQL based in the ERD designed on day 1. This section explains the activities and progress made during the second day.

Create a database

```
CREATE DATABASE IF NOT EXISTS de_gouden_tarwe
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;
```

This database contains 24 tables: region, cities, branches, product_categories, products, suppliers, ingredients, product_recipes, customers, employees, employee_shifts, orders, order_items, inventory_stock, purchase_orders, purchase_order_items, promotions, customer_feedback, daily_production, waste_log, branch_operating_hours, price_history, loyalty_transactions, and equipment.

When creating tables, parent tables should be created first because they do not contain foreign keys. After that, child tables that contain foreign keys can be created.

Before creating tables, we need to specify the database where the tables will be created.

```
USE de_gouden_tarwe;
```

Create the region table

```
DROP TABLE IF EXISTS regions;
CREATE TABLE regions (
    region_id          INT AUTO_INCREMENT PRIMARY KEY,
    region_name        VARCHAR(100) NOT NULL UNIQUE,
    country            VARCHAR(50) NOT NULL DEFAULT 'Nederland'
) ENGINE=InnoDB;
```

Create the cities table

```
DROP TABLE IF EXISTS cities;
CREATE TABLE cities (
    city_id            INT AUTO_INCREMENT PRIMARY KEY,
    city_name          VARCHAR(100) NOT NULL,
    latitude           DECIMAL(8,4),
    longitude          DECIMAL(8,4),
    region_id          INT NOT NULL,
    CONSTRAINT fk_cities_region
        FOREIGN KEY (region_id) REFERENCES regions(region_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Create the branches table

```
DROP TABLE IF EXISTS branches;
CREATE TABLE branches (
    branch_id          INT AUTO_INCREMENT PRIMARY KEY,
    branch_name        VARCHAR(200) NOT NULL,
    city_id            INT NOT NULL,
    address            VARCHAR(300) NOT NULL,
    postal_code        VARCHAR(10) NOT NULL,
    phone              VARCHAR(20),
    email              VARCHAR(200),
    opened_date        DATE NOT NULL,
    size_category       VARCHAR(20) CHECK (size_category IN ('Klein',
'Medium', 'Groot')),
    seating_capacity    INT DEFAULT 0 CHECK (seating_capacity >= 0),
    is_active          TINYINT(1) DEFAULT 1,
    latitude           DECIMAL(10,6),
    longitude          DECIMAL(10,6),
    CONSTRAINT fk_branches_city
        FOREIGN KEY (city_id) REFERENCES cities(city_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

Create the product_categories table

```
DROP TABLE IF EXISTS product_categories;
CREATE TABLE product_categories (
    category_id        INT AUTO_INCREMENT PRIMARY KEY,
    category_name       VARCHAR(100) NOT NULL,
    description         TEXT,
```

```

        is_active          BOOLEAN DEFAULT TRUE,
        parent_category_id INT,
        CONSTRAINT fk_product_categories_parent_category
            FOREIGN KEY (parent_category_id) REFERENCES
product_categories(category_id)
            ON DELETE SET NULL ON UPDATE CASCADE
    ) ENGINE=INNODB;

```

Create the branch_operating_hours table

```

DROP TABLE IF EXISTS branch_operating_hours;
CREATE TABLE branch_operating_hours (
    operating_hours_id INT AUTO_INCREMENT PRIMARY KEY,
    branch_id          INT NOT NULL,
    day_of_week        INT CHECK (day_of_week >= 0 AND day_of_week <=
6),
    day_name           ARCHAR(20) NOT NULL,
    opening_time       TIME,
    closing_time       TIME,
    is_open            BOOLEAN DEFAULT TRUE,
    CONSTRAINT fk_branch_hours_branch
        FOREIGN KEY (branch_id) REFERENCES branches(branch_id)
            ON DELETE CASCADE, UNIQUE KEY uq_oh (branch_id, day_of_week)
) ENGINE=INNODB;

```

Create the products table

```

DROP TABLE IF EXISTS products;
CREATE TABLE products (
    product_id          INT AUTO_INCREMENT PRIMARY KEY,
    sku                 VARCHAR(20) NOT NULL UNIQUE,
    product_name         VARCHAR(200) NOT NULL,
    category_id         INT NOT NULL,
    unit_price          DECIMAL(10,2) NOT NULL CHECK(unit_price > 0),
    weight_kg           DECIMAL(6,3),
    shelf_life_days     INT CHECK (shelf_life_days > 0),
    is_year_round       BOOLEAN DEFAULT TRUE,
    allergens           VARCHAR(500),
    cost_price          DECIMAL(10,2) NOT NULL CHECK (cost_price > 0),
    currency            CHAR(3) DEFAULT 'EUR',
    is_active           BOOLEAN DEFAULT TRUE,
    created_at          TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_products_category
        FOREIGN KEY (category_id)
            REFERENCES product_categories(category_id)
            ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

```

Create the suppliers table

```

DROP TABLE IF EXISTS suppliers;
CREATE TABLE suppliers (
    supplier_id          INT AUTO_INCREMENT PRIMARY KEY,
    supplier_name        VARCHAR(200) NOT NULL,
    address             VARCHAR(300),
    postal_code         VARCHAR(10),
    city               VARCHAR(100),
    phone              VARCHAR(20),
    email              VARCHAR(200),
    specialty          VARCHAR(200),
    vat_number         VARCHAR(20) UNIQUE,

```

```

        is_active                BOOLEAN DEFAULT TRUE,
        payment_terms            VARCHAR(20),
        currency                 CHAR(3) DEFAULT 'EUR',
        contract_start_date      DATE
    ) ENGINE=InnoDB;

```

Create the ingredients table

```

DROP TABLE IF EXISTS ingredients;
CREATE TABLE ingredients (
    ingredient_id                INT AUTO_INCREMENT PRIMARY KEY,
    ingredient_name              VARCHAR(200) NOT NULL,
    unit_of_measure              VARCHAR(20) NOT NULL,
    cost_per_unit                DECIMAL(10,2) CHECK(cost_per_unit > 0),
    storage_type                 VARCHAR(20) CHECK(storage_type IN
('Droog', 'Gekoeld', 'Vers', 'Diepvries')),
    primary_supplier_id         INT NULL,
    min_stock_level              INT NOT NULL,
    reorder_quantity             INT NOT NULL,
    is_active                    BOOLEAN DEFAULT TRUE,
    CONSTRAINT fk_ingredients_supplier
        FOREIGN KEY (primary_supplier_id)
        REFERENCES suppliers(supplier_id)
        ON DELETE SET NULL ON UPDATE CASCADE
    ) ENGINE=InnoDB;

```

Create the product_recipes table

```

DROP TABLE IF EXISTS product_recipes;
CREATE TABLE product_recipes (
    recipe_id                    INT AUTO_INCREMENT PRIMARY KEY,
    product_id                   INT NOT NULL,
    ingredient_id                 INT NOT NULL,
    quantity_needed              DECIMAL(8,3) CHECK(quantity_needed > 0),
    unit_of_measure              VARCHAR(20) NOT NULL,
    notes                         TEXT,
    CONSTRAINT fk_productrecipes_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_productrecipes_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredients(ingredient_id)
        ON DELETE RESTRICT
    ) ENGINE=InnoDB;

```

Create the price_history table

```

DROP TABLE IF EXISTS price_history;
CREATE TABLE price_history (
    price_history_id             INT AUTO_INCREMENT PRIMARY KEY,
    product_id                   INT NOT NULL,
    old_price                     DECIMAL(10,2) NOT NULL,
    new_price                     DECIMAL(10,2) NOT NULL,
    effective_date                DATE NOT NULL,
    change_reason                 VARCHAR(200),
    CONSTRAINT fk_pricehistory_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE CASCADE
    ) ENGINE=InnoDB;

```

Create the customers table

```
DROP TABLE IF EXISTS customers;
CREATE TABLE customers (
    customer_id          INT AUTO_INCREMENT PRIMARY KEY,
    first_name           VARCHAR(100) NOT NULL,
    last_name            VARCHAR(100) NOT NULL,
    email                VARCHAR(200) NOT NULL UNIQUE,
    phone                VARCHAR(20) NULL,
    date_of_birth        DATE NULL,
    gender               CHAR(1) CHECK(gender IN('M', 'V', 'X')),
    address              VARCHAR(300),
    postal_code          VARCHAR(10),
    city_id              INT NULL,
    join_date            DATE NOT NULL,
    is_loyalty_member    BOOLEAN DEFAULT FALSE,
    loyalty_tier          VARCHAR(20) CHECK(loyalty_tier IN('Brons',
'Zilver', 'Goud', 'Platina')),
    loyalty_points        INT DEFAULT 0 CHECK (loyalty_points >= 0),
    newsletter_opt_in    BOOLEAN DEFAULT FALSE,
    preferred_branch_id  INT NULL,
    CONSTRAINT fk_customers_city
        FOREIGN KEY (city_id)
        REFERENCES cities(city_id)
        ON DELETE SET NULL,
    CONSTRAINT fk_customers_branch
        FOREIGN KEY (preferred_branch_id)
        REFERENCES branches(branch_id)
        ON DELETE SET NULL
) ENGINE=InnoDB;
```

Create the employees table

```
DROP TABLE IF EXISTS employees;
CREATE TABLE employees (
    employee_id          INT AUTO_INCREMENT PRIMARY KEY,
    first_name           VARCHAR(100) NOT NULL,
    last_name            VARCHAR(100) NOT NULL,
    email                VARCHAR(200) UNIQUE,
    phone                VARCHAR(20),
    date_of_birth        DATE NOT NULL,
    branch_id            INT NOT NULL,
    position_title       VARCHAR(100) NOT NULL,
    department           VARCHAR(50) NOT NULL,
    position_level       INT CHECK(position_level >= 1 AND
position_level <=7),
    monthly_salary        DECIMAL(10,2) CHECK(monthly_salary > 0),
    contract_type        VARCHAR(20) CHECK(contract_type IN('Vast',
'Tijdelijk', 'Oproep', 'Stage')),
    weekly_hours         INT CHECK(weekly_hours >= 0),
    hire_date            DATE NOT NULL,
    termination_date     DATE,
    is_active            BOOLEAN DEFAULT TRUE,
    manager_id           INT,
    CONSTRAINT fk_employees_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_employee_manager
        FOREIGN KEY (manager_id)
        REFERENCES employees(employee_id)
```

```
        ON DELETE SET NULL
    ) ENGINE=InnoDB;
```

Create the employee_shifts table

```
DROP TABLE IF EXISTS employee_shifts;
CREATE TABLE employee_shifts (
    shift_id            INT AUTO_INCREMENT PRIMARY KEY,
    employee_id         INT NOT NULL,
    branch_id           INT NOT NULL,
    start_datetime      DATETIME NOT NULL,
    end_datetime        DATETIME NOT NULL,
    shift_type          VARCHAR(20) CHECK(shift_type IN('Ochtend',
'Middag', 'Avond', 'Hele dag')),
    hours_worked        DECIMAL(4,2) CHECK(hours_worked >= 0),
    status              VARCHAR(20) CHECK(status IN('Gepland',
'Gewerkt', 'Ziek', 'Verlof')),
    CONSTRAINT fk_employeeeshift_employee
        FOREIGN KEY (employee_id)
        REFERENCES employees(employee_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_employeeeshift_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Create the orders table

```
DROP TABLE IF EXISTS orders;
CREATE TABLE orders (
    order_id            INT AUTO_INCREMENT PRIMARY KEY,
    branch_id           INT NOT NULL,
    customer_id         INT NULL,
    employee_id         INT NULL,
    order_datetime      DATETIME NOT NULL,
    order_channel       VARCHAR(20) CHECK(order_channel
IN('Winkel', 'Online', 'Telefoon', 'Catering')),
    order_status        VARCHAR(20) CHECK(order_status
IN('Voltooid', 'Geannuleerd', 'In Behandeling', 'Terugbetaald')),
    payment_method      VARCHAR(20) NOT NULL,
    subtotal            DECIMAL(10,2) NOT NULL,
    vat_amount          DECIMAL(10,2) NOT NULL,
    total_amount        DECIMAL(10,2) CHECK(total_amount >=
0),
    loyalty_points_earned INT DEFAULT 0,
    notes              TEXT NULL,
    CONSTRAINT fk_orders_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE SET NULL ,
    CONSTRAINT fk_order_employee
        FOREIGN KEY (employee_id)
        REFERENCES employees(employee_id)
        ON DELETE SET NULL
) ENGINE=InnoDB;
```

Create the order_items table

```
DROP TABLE IF EXISTS order_items;
CREATE TABLE order_items (
    order_item_id          INT AUTO_INCREMENT PRIMARY KEY,
    order_id               INT NOT NULL,
    product_id             INT NOT NULL,
    quantity               INT NOT NULL CHECK(quantity > 0),
    unit_price             DECIMAL(10,2) DEFAULT 0,
    discount_amount        DECIMAL(10,2) DEFAULT 0,
    line_total             DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_orderitem_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_orderitem_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Create the promotions table

```
DROP TABLE IF EXISTS promotions;
CREATE TABLE promotions (
    promotion_id           INT AUTO_INCREMENT PRIMARY KEY,
    promotion_name         VARCHAR(200) NOT NULL,
    description            TEXT,
    discount_type          VARCHAR(20) CHECK(discount_type
IN('percentage', 'fixed_amount', 'points_multiplier')),
    discount_value         DECIMAL(10,2) NOT NULL,
    start_date            DATE NOT NULL,
    end_date              DATE NOT NULL,
    branch_id             INT NULL,
    is_active             BOOLEAN DEFAULT TRUE,
    times_used            INT DEFAULT 0,
    CONSTRAINT fk_promotions_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE SET NULL,
    CONSTRAINT chk_promo_dates CHECK (end_date >= start_date)
    # ketentuannya adalah tanggal berakhir harus lebih lama atau sama
    dengan tanggal mulai
) ENGINE=InnoDB;
```

Create the customer_feedback table

```
DROP TABLE IF EXISTS customer_feedback;
CREATE TABLE customer_feedback (
    feedback_id           INT AUTO_INCREMENT PRIMARY KEY,
    order_id             INT NULL,
    customer_id          INT NULL,
    branch_id            INT NOT NULL,
    rating               INT NOT NULL CHECK(rating >= 1 AND rating
<= 5),
    comment              TEXT NULL,
    feedback_category    VARCHAR(50),
    feedback_date        DATE NOT NULL,
    CONSTRAINT fk_custfeedback_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
```



```

        ON DELETE SET NULL,
    CONSTRAINT fk_custfeedback_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE SET NULL,
    CONSTRAINT fk_custfeedback_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

Create the loyalty_transaction table

```

DROP TABLE IF EXISTS loyalty_transaction;
CREATE TABLE loyalty_transaction (
    loyalty_transaction_id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    transaction_type VARCHAR(20) CHECK(transaction_type
IN('Verdiend', 'Ingewisseld', 'Bonus', 'Verlopen')),
    points INT NOT NULL,
    transaction_date DATE NOT NULL,
    order_id INT NULL,
    description VARCHAR(200),
    CONSTRAINT fk_loyaltytransaction_customer
        FOREIGN KEY (customer_id)
        REFERENCES customers(customer_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_loyaltytransaction_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id)
        ON DELETE SET NULL
) ENGINE=InnoDB;

```

Create the purchase_orders table

```

DROP TABLE IF EXISTS purchase_orders;
CREATE TABLE purchase_orders (
    purchase_order_id INT AUTO_INCREMENT PRIMARY KEY,
    branch_id INT NOT NULL,
    supplier_id INT NOT NULL,
    po_number VARCHAR(20) NOT NULL UNIQUE,
    order_date DATE NOT NULL,
    delivery_date DATE NULL,
    status VARCHAR(20) CHECK(status IN('Geleverd',
'Besteld', 'Deels Geleverd', 'Geannuleerd')),
    total_amount DECIMAL(12,2) NOT NULL,
    currency CHAR(3) DEFAULT 'EUR',
    notes TEXT,
    CONSTRAINT fk_poorders_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_poorders_supplier
        FOREIGN KEY (supplier_id)
        REFERENCES suppliers(supplier_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

Create the purchase_order_items table

```

DROP TABLE IF EXISTS purchase_order_items;

```

```

CREATE TABLE purchase_order_items (
    po_item_id          INT AUTO_INCREMENT PRIMARY KEY,
    purchase_order_id    INT NOT NULL,
    ingredient_id        INT NOT NULL,
    quantity            DECIMAL(10,1) CHECK(quantity > 0),
    unit_of_measure      VARCHAR(20) NOT NULL,
    unit_cost            DECIMAL(10,2) NOT NULL,
    line_total           DECIMAL(10,2) NOT NULL,
    CONSTRAINT fk_poitems_poroder
        FOREIGN KEY (purchase_order_id)
        REFERENCES purchase_orders(purchase_order_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_poitems_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredients(ingredient_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

Create the inventory_stock table

```

DROP TABLE IF EXISTS inventory_stock;
CREATE TABLE inventory_stock (
    inventory_id        INT AUTO_INCREMENT PRIMARY KEY,
    branch_id           INT NOT NULL,
    ingredient_id       INT NOT NULL,
    current_quantity    DECIMAL(10,2) NOT NULL,
    minimum_quantity    DECIMAL(10,2) NOT NULL,
    maximum_quantity    DECIMAL(10,2) NOT NULL,
    unit_of_measure      VARCHAR(20) NOT NULL,
    last_update         DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_inventorystock_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE CASCADE,
    CONSTRAINT fk_inventorystock_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredients(ingredient_id)
        ON DELETE RESTRICT
) ENGINE=InnoDB;

```

Create the waste_log table

```

DROP TABLE IF EXISTS waste_log;
CREATE TABLE waste_log (
    waste_id            INT AUTO_INCREMENT PRIMARY KEY,
    branch_id           INT NOT NULL,
    product_id          INT NOT NULL,
    waste_date          DATE NOT NULL,
    quantity            INT NOT NULL CHECK(quantity > 0),
    reason              VARCHAR(50) NOT NULL,
    cost_value          DECIMAL(10,2) NOT NULL,
    recorded_by_employee_id INT NULL,
    CONSTRAINT fk_wastelog_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_wastelog_product
        FOREIGN KEY (product_id)
        REFERENCES products(product_id)
        ON DELETE RESTRICT,
    CONSTRAINT fk_wastelog_employee

```

```
        FOREIGN KEY (recorded_by_employee_id)
        REFERENCES employees(employee_id)
        ON DELETE SET NULL
    ) ENGINE=InnoDB;
```

Create the equipment table

```
DROP TABLE IF EXISTS equipment;
CREATE TABLE equipment (
    equipment_id          INT AUTO_INCREMENT PRIMARY KEY,
    branch_id             INT NOT NULL,
    equipment_name        VARCHAR(200) NOT NULL,
    department            VARCHAR(50) NOT NULL,
    serial_number         VARCHAR(50) UNIQUE,
    purchase_date         DATE NOT NULL,
    purchase_cost         DECIMAL(12,2) NOT NULL,
    status                VARCHAR(30) CHECK(status IN('In
Onderhoud', 'Buiten Gebruik', 'Operationeel')),
    last_maintenance_date DATE,
    next_maintenance_date DATE,
    CONSTRAINT fk_equipment_branch
        FOREIGN KEY (branch_id)
        REFERENCES branches(branch_id)
        ON DELETE CASCADE
) ENGINE=InnoDB;
```